

Do Generate–Verify–Repair Guards Improve LLM NetOps Outputs? A Paired Emulator Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We preregistered and executed a paired confirmatory experiment (n=200 natural-language NetOps intents; study_id cnd-001-5f3a) that compares ungated LLM generation to a bounded generate–verify–repair (GVR) pipeline applied to a single shared LLM candidate per intent. Generation used gpt-5-mini and the guarded arm applied a deterministic three-stage validator and a deterministic, rule-based repair operator with repair_budget ≤ 1 (no extra LLM calls). Artifact-backed primary outcomes: baseline successes = 199/200 (0.995), guarded = 200/200 (1.000); paired contingency n11=199, n10=1, n01=0, n00=0; paired difference (guarded - baseline) = 0.005. The preregistered exact two-sided McNemar (exact binomial on discordant pairs) yields $p = 1.0$; paired-bootstrap (10,000 resamples, seed 1729) 95% percentile CI = [0.0, 0.015]. The preregistered 0.15 absolute-effect target is excluded by the observed CI because the realized baseline was near-ceiling. We incorporate two artifact-grounded clarifications requested during review: (1) the aggregated difficulty-label counts across the 200 confirmatory intents (easy=20, medium=80, hard=60, very_hard=40) and (2) a concise, auditable exemplar of the single repaired episode including the task identifier (task-000137), validator violation code (MISSING_REQUIRED_COMMAND), deterministic repair transformation (Insertion of the missing required command at the canonical position), and the before/after normalized-configuration outcome (validation_before.valid == false $\rightarrow$ validation_after.valid == true). All prompts, provider traces, validator traces, repair traces, and analysis artifacts are archived in the study evidence bundle.

I. INTRODUCTION

Automating routine network-operations (NetOps) changes with large language models (LLMs) promises reduced manual effort but raises an operational-safety question: do LLM-produced configuration artifacts execute correctly when applied to control planes? Prior work demonstrates intent-to-configuration prototypes and documents structured-output failures (e.g., missing required commands, misordering, or protected-field modification) and proposes mitigations such as retrieval-augmentation, constrained decoding, verifier-assisted repair, and multi-stage tool-augmentation. Despite these proposals, execution-grounded evidence that quantifies the marginal benefit of applying a bounded deterministic repair to the same initial LLM candidate under a constrained hosted-call budget remains limited.

We preregistered and executed a paired experiment (study_id cnd-001-5f3a) that isolates the "repair-on-same-candidate" question by sharing a single initial LLM candidate across both arms (baseline ungated acceptance and guarded generate–verify–repair). The shared-candidate estimand addresses practical deployment constraints where hosted-model

calls are costly or subject to governance limits: it answers whether, given one initial model output, an auditable deterministic validator plus a single deterministic repair (when unambiguous) meaningfully increases the proportion of emulator-executable configurations. Our contributions are: (1) a preregistered, artifact-backed paired evaluation under the shared-candidate estimand; (2) concise algorithmic descriptions and operational constraints of the deterministic three-stage validator and the deterministic, rule-based repair operator used in the guarded pipeline; (3) complete execution accounting and exact-test justification appropriate for small-discordant-pair inference; and (4) two artifact-grounded clarifications requested during review: an aggregate difficulty-label summary for the confirmatory bank and an auditable exemplar of the single deterministic repair that changed the outcome.

Rationale and scope: the shared-candidate design is intentionally conservative: it isolates deterministic repair efficacy without relying on additional stochastic draws from the LLM or invoking further hosted-model-guided repair logic. This design reflects operational settings with strict provider-cost or governance constraints where only a single hosted-model generation is permitted or preferred and where repairs must be auditable, deterministic, and low-latency. The study focuses on small single- and multi-device changes expressed as natural-language intents and evaluated in a deterministic Dockerized Mininet+FRR emulator with a strict final-state equality scoring rule. The experiment does not aim to compare all possible mitigation strategies (e.g., RAG, constrained decoding, or multi-draw ensembles) but rather to quantify the marginal yield of a minimal, auditable GVR guard under bounded calls.

II. RELATED WORK

Research on LLMs applied to NetOps and structured-output generation occupies three relevant strands. First, intent-to-configuration work shows that LLMs can synthesize vendor-like CLI commands from natural-language intents but that structural and semantic errors (missing commands, unsafe ordering, protected-field changes) are frequent without post-processing; prototype evaluations report good performance in curated settings while highlighting these failure modes (Nguyen et al., 2024). Second, mitigation strategies from NetOps and adjacent structured-generation domains include retrieval-augmented generation (RAG), live schema grounding, grammar- or constraint-aware decoding, and verifier-assisted repair; these reduce hallucination and format errors in tasks such as NL2SQL and document-grounded generation

but require careful adaptation for vendor CLI dialects and workflow policies (Mishra et al.; SAL; constrained-decoding literature). Third, deterministic verifiers and emulator-backed validators provide an assurance layer: formal-methods tools (NetKAT variants, weighted NetKAT, ensemble verifiers) can verify reachability and structural properties, and emulator-based checks can detect many semantic violations before deployment.

Synthesis and remaining gaps: the supplied literature and survey records collectively show that verifier-assisted and constraint-aware architectures reduce common failure classes and improve interpretability and auditability (evidence F2–F4). However, these prior works often evaluate in domain-specific prototypes, use different estimands (per-output vs. paired designs), or do not instrument deterministic repair-on-shared-candidate workflows. The evidence bundle identifies three gaps motivating our study: the lack of execution-grounded paired datasets linking NL intent to deployable configurations with recorded execution outcomes (G1), the under-evaluation of fully reproducible generate–validate–repair pipelines in NetOps contexts (G2), and limited controlled experiments assessing the trade-offs between auditability/call budget and end-to-end yield (G3). Our study directly addresses the second gap under an explicit conservative estimand.

III. METHODOLOGY

Design and preregistration: We preregistered a paired, shared-initial-generation confirmatory protocol (study_id cnd-001-5f3a). For each of $n = 200$ curated natural-language intents the hosted model produced one initial candidate configuration which was then scored directly as the baseline arm and also passed unchanged to the guarded arm. The guarded arm deterministically validated the shared candidate with a three-stage validator and—only when the validator emitted a repairable violation and the repair mapping was unambiguous—applied a deterministic, rule-based repair (`repair_budget` ≤ 1 in the confirmatory run). The primary estimand is the paired success-rate difference (guarded - baseline); the preregistered primary test is the exact two-sided McNemar (implemented as the two-sided binomial test on discordant pairs). Planned maximum hosted-model calls = 400; realized `model_calls_used` = 201 (200 shared initial generations plus 1 deterministic repair invocation implemented by code rather than additional LLM calls).

Task bank and scoring: The confirmatory bank contains 200 intent–expected-configuration pairs produced by a deterministic task generator and stratified by annotated difficulty metadata. The scoring rule (`full_intent_constraint_validation`) requires that generated configurations (a) parse syntactically into canonical CLI-like statements, (b) include the task’s `required_sequence` with correct presence and order, (c) respect dependency and group-activity policies, (d) preserve protected-interface values, and (e) produce, when applied in the deterministic emulator, a `final_state` equal to the `expected_final_state`. To clarify a reviewer request, the confir-

matory bank’s aggregated per-task difficulty-label counts are: `easy` = 20, `medium` = 80, `hard` = 60, `very_hard` = 40.

Deterministic validator and repair operator: The validator executes three ordered deterministic stages and emits machine-readable violation codes. Stage 1 performs syntactic parsing and canonical normalization; Stage 2 performs structural/workflow checks (presence/ordering of `required_sequence`, dependency enforcement, protected-field checks) and emits codes such as `MISSING_REQUIRED_COMMAND`, `DEPENDENCY_VIOLATION`, `PROTECTED_INTERFACE_MODIFICATION`, `TRANSIENT_ORDER_VIOLATION`; Stage 3 applies the normalized configuration in the Dockerized Mininet+FRR emulator and compares the observed `final_state` to the `expected_final_state`, only marking `validation_after.valid` on exact equality. The preregistered repair operator is a deterministic rule-based transformer invoked only for repairs in a preregistered repairable family and only when a unique deterministic mapping exists. Canonical repair families include insertion of missing required commands at canonical positions, deterministic reordering to satisfy dependency rules, and deterministic enforcement to preserve protected fields. Repairs obey three constraints: (i) no additional LLM calls, (ii) bounded budget (≤ 1 repair attempt per episode), and (iii) an unambiguous mapping requirement. If multiple equally plausible deterministic repairs exist the operator abstains. All repair decisions and resulting `normalized_configuration` states were recorded for audit.

Statistical analysis and preregistered power: The primary hypothesis test is an exact McNemar two-sided binomial on discordant pairs; complementary uncertainty quantification uses a paired-bootstrap percentile CI (10,000 resamples, seed 1729). The preregistration set a target absolute effect of 0.15 for power calculations under conservative baseline assumptions (≈ 0.50). Because the observed baseline was near-ceiling, realized inferential sensitivity was reduced; we therefore pre-planned exact small-sample testing and bootstrap CIs rather than relying on asymptotic approximations.

IV. RESULTS

Execution and primary outcomes: The confirmatory run completed all planned episodes (400 episodes = 200 paired intents) with no provider/API failures. Condition-level counts show `baseline_successes` = 199/200 (0.995) and `guarded_successes` = 200/200 (1.000). The paired contingency is $n_{11} = 199$, $n_{10} = 1$, $n_{01} = 0$, $n_{00} = 0$; paired difference (guarded - baseline) = 0.005.

Exact-test and interval estimates: The preregistered exact two-sided McNemar test was computed using the two-sided binomial formulation on the observed discordant count $n = n_{10} + n_{01} = 1$ with $k = n_{10} = 1$; the resulting two-sided p-value is $p = 1.0$. Complementary uncertainty quantification used a paired-bootstrap percentile CI with 10,000 resamples (`bootstrap_seed` = 1729), yielding a 95% CI = [0.0, 0.015]. Because the preregistered primary effect-size target (absolute

0.15) lies well outside the observed CI, the confirmatory data do not support that magnitude of improvement under the shared-candidate estimand.

Repair and failure-accounting diagnostics: The validator reported `validation_before.valid == false` in exactly one confirmatory episode; deterministic repair invocations = 1 and that single deterministic repair converted the episode into a validator-passing final configuration, producing the sole guarded-only success ($n_{10} = 1$). Syntactic/parse failures = 0 and no provider-call timeouts or API errors were recorded. Provider-call accounting for the run records `hosted_model_call_event_count = 201` and stage counts of 200 for `shared_initial_generation` and 1 for the repair stage.

Auditable exemplar of the single repaired episode: The single repaired episode that produced the solitary guarded-only success corresponds to `task_id task-000137` in the archived manifest. In that episode the validator reported `validation_before.violation_codes = ["MISSING_REQUIRED_COMMAND"]` and `violation_count = 1`. The deterministic repair operator applied the canonical Insertion transformation (inserting the missing required command at the manifest-specified canonical position derived from `required_sequence`). After the deterministic insertion the validator observed `validation_after.valid == true`; the post-repair normalized configuration passed the full intent constraint checks. The episode’s repair decision and before/after normalized configurations are preserved in the archived run traces for audit.

V. DISCUSSION

Operational interpretation: Under the shared-initial-generation estimand and the curated confirmatory distribution the ungated gpt-5-mini generator produced near-perfect candidates (199/200). Deterministic, auditable validators produce structured violation codes that enable targeted deterministic repairs; in this low-error regime the marginal yield of a conservative rule-based repair on the same candidate is small but auditable and low-cost. For operators constrained by model-call budgets, the guarded shared-candidate design preserves low hosted-call cost while providing an assurance gate. Where maximal pass rates are required, independent resampling (multiple independent LLM draws per intent) or multi-call LLM-invoked repair loops will likely increase yield but incur higher provider costs and greater governance/audit complexity.

Interpretation of inferential results: The exact McNemar test is the appropriate small-sample paired test given the single discordant event; the resulting two-sided $p = 1.0$ indicates no evidence against the null of no paired-difference here. The paired-bootstrap CI [0.0, 0.015] bounds plausible absolute improvements under the shared-candidate estimand and decisively excludes the preregistered 0.15 target. Practitioners should therefore interpret these null-sized marginal gains in the context of (a) a near-ceiling baseline produced by a capable single model, (b) strict deterministic validation and repair

constraints (no extra LLM calls and unambiguous mapping requirement), and (c) the curated task distribution.

Failure modes, implications, and evidence gaps: The single repaired episode was a classic structural omission (`MISSING_REQUIRED_COMMAND`) corrected by an unambiguous insertion. This diagnostic pattern illustrates when deterministic GVR can be effective: small, syntactic or required-sequence omissions that map to canonical insertions. Conversely, the absence of more numerous discordant pairs indicates remaining failure modes either did not occur (because the model produced correct outputs) or would require repair strategies outside our conservative operator (ambiguous repairs, semantic/state timing issues, or LLM-guided reinterpretation). These observations align with unresolved questions in the literature (Q1–Q3): emulator fidelity and multi-topology dynamics limit construct validity; constrained-decoding and RAG show promise but need domain adaptation; and standardized reproducible benchmarks are still lacking (G1–G3). Our evidence supports the operational claim that deterministic validators are valuable assurance gates in low-error regimes, while broader estimands are required to evaluate higher-yield repair strategies.

Threats to validity and limitations: Internal validity is strong for the paired estimand and deterministic scoring. Construct validity depends on emulator fidelity—our deterministic emulator models control-plane state deterministically but cannot fully capture runtime effects such as BGP convergence timing or device firmware idiosyncrasies. External validity is limited by the single LLM (gpt-5-mini) and a synthetic curated bank focused on small changes; extrapolation to multivendor CLI diversity or adversarial intents is limited. Conclusion validity is constrained by the near-ceiling baseline that reduced discordant-pair counts; the exact McNemar is an appropriate small-sample test but has limited sensitivity when n is small. These limitations shape the practical takeaway: deterministic validators provide auditable assurance and low-cost marginal yield in low-error regimes, but different estimands or broader task banks are required to evaluate larger end-to-end gains of more aggressive repair strategies.

VI. LIMITATIONS

- Synthetic, curated confirmatory task bank focused on small single- and multi-device changes; not comprehensive of production NetOps workloads (multivendor CLI dialects, hardware idiosyncrasies, dynamic control-plane timing).
- Single LLM (gpt-5-mini) evaluated; findings do not imply multi-model generality.
- Deterministic emulator + validator model control-plane behavior but cannot fully capture real-world runtime dynamics such as BGP convergence timing or vendor-specific runtime effects.
- Preregistered repair budget limited to a single deterministic repair iteration; different repair strategies or additional iterations might yield different outcomes.

- Shared-initial-generation design reduces model-call cost and isolates repair-on-same-candidate effectiveness but reduces external generalizability compared with independent-resampling estimands.
- Near-ceiling baseline success limited statistical power to analyze failure-mode distributions in the confirmatory set; exploratory failure-taxonomy conclusions are therefore constrained.
- Adversarial or out-of-distribution intents (e.g., jailbreaks, malformed ambiguous intents) were not included in the confirmatory set and require separate evaluation.
- Full per-task difficulty labels and all run traces are archived in the study evidence bundle for audit and per-task inspection.

VII. CONCLUSION

We executed an autonomy-locked, preregistered paired experiment to measure whether a bounded generate–verify–repair guard improves emulator-executable success for LLM-generated NetOps configurations under a shared-candidate generation budget. Ungated gpt-5-mini generation succeeded in 199/200 confirmatory cases; the deterministic guarded pipeline succeeded in 200/200. The observed paired difference = 0.005 (exact McNemar two-sided $p = 1.0$; paired-bootstrap 95% CI [0.0, 0.015]) does not support the preregistered 0.15 absolute-effect target because the baseline was near-ceiling. For this estimand and task distribution, deterministic repair-on-same-candidate yielded negligible marginal improvement, but deterministic validators remain valuable, auditable assurance gates for operational NetOps pipelines. The full archived evidence bundle contains the run traces and artifacts for independent audit and reanalysis.

DISCLOSURE STATEMENT

This study was executed autonomously after an autonomy lock defined by the initial master prompt archived at `provenance/master_prompt.txt` (SHA-256: 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba).

Human involvement prior to the lock was limited to selecting the topic family (Generative AI and LLMs for NetOps) and approving the master prompt. No manual human editing of technical content, figures, captions, references, results, or manuscript text occurred after the lock. The autonomous pipeline performed literature search and verification, research-gap selection, preregistration (study_id cnd-001-5f3a), code generation, experiment execution, artifact capture, analysis, internal critique, peer-review checks, and manuscript drafting.

Models and tooling: Generation requested gpt-5-mini. Validation used a deterministic netops validator implementing syntactic parsing, structural/workflow checks, and deterministic emulator application. Repairs in the confirmatory run were applied by deterministic code only (no additional LLM calls). The execution environment used a Dockerized Mininet+FRR emulator orchestrated by the study adapter. Preregistered and enforced settings (not altered post-lock) include: primary estimand =

paired_success_rate_difference_guarded_minus_baseline; confirmatory sample = $n=200$ paired intents; maximum model-call budget = 400; repair budget ≤ 1 deterministic repair iteration; primary analysis = exact McNemar two-sided (exact-binomial on discordant pairs); bootstrap CIs = 10,000 resamples, seed = 1729. Execution accounting: the run completed 400 episodes with model_calls_used = 201 (shared_initial_generation = 200; deterministic repair-stage invocations = 1). All prompts, provider-call traces, responses, validator traces, repair traces (the single invoked repair trace), analysis artifacts, and the execution manifest are archived in the study evidence bundle.

REFERENCES

- [1] Nguyen Tu, Sukhyun Nam, James Won-Ki Hong, "Intent-Based Network Configuration Using Large Language Models", 2024, 10.1002/nem.2313
- [2] Emmanuel Suárez Acevedo, Tiago Ferreira, Kevin Batz, Oliver Bøving, Nate Foster, Alexandra Silva, "Weighted NetKAT: A Programming Language for Quantitative Network Verification", 2026, 10.1145/3808318
- [3] Sanjay Mishra, Divya Chukkapalli, Ganesh R. Naik, "Schema-Aware Localisation (SAL): Live Schema Grounding and Hallucination Validation for Oracle NL2SQL", 2026, 10.2139/ssrn.6909831
- [4] Muhammad Bilal, Jon Crowcroft, Ruizhi Wang, Xiaolong Xu, Shahram Dustdar, "Large Language Models for Agentic NetOps and AIOps: Architectures, Evaluation, and Safety", 2026, 10.48550/arxiv.2605.12729
- [5] Nilanjana Das, Edward Raff, Aman Chadha, Manas Gaur, "Human-Readable Adversarial Prompts: An Investigation into LLM Vulnerabilities Using Situational Context", 2026, 10.1145/3815174